D597 Task 2: Non-Relational Database Design and Implementation

Part 1: Design

A. Select one of the provided scenarios and complete the following:

For Task 2, I have selected Scenario 1, HealthFit Innovations, a rapidly growing healthcare technology company working to revolutionize healthcare data collection, integration, and analytics with a new platform called "HealthTrack".

HealthTrack accesses data from wearable devices, electronic health records (EHRs), medical imaging systems, and patient-reported outcomes to enable real-time monitoring, predictive analytics, and personalized health insights. As the user base grows and the variety of data increases, HealthFit Innovations' current database infrastructure is becoming inadequate. They require a new database that is scalable to their needs and has a performance that matches.

1. Describe a business problem that can be solved with a database solution and is in alignment with the chosen scenario.

HealthTrack's current database management system (DBMS) struggles to meet performance demands. It is expected to become a performance bottleneck as the volume and variety of real-time health data grows. This limitation will hinder business data integration and analysis efforts. Possibly increasing delivery times of personalized insights and impacting user satisfaction.

Switching to a NoSQL database gives HealthFit the flexibility and speed needed to keep up with its growing data demands. It scales easily, handles different data types without a rigid structure, and runs real-time queries more efficiently. This change boosts system performance and makes it easier to dig into complex datasets and uncover trends that can shape better business decisions and improve how the platform serves its users.

The following are three business problems I will solve with this database solution:

1. Understanding Health Trends Among Users

a. Is there a correlation between users' health status and their likelihood of using tracking devices?
b. Are healthier patients more engaged with wearable tracking devices?

2. Device Brand Preference and Support Strategy
    a. Which brands are most and least preferred by users?
    b. Should HealthTrack reduce or discontinue support for less preferred brands?
    c. Which brands should HealthTrack prioritize for integration and feature support?
3. Product Price and Customer Sentiment
    a. Do higher-priced devices correlate with better user reviews and satisfaction?
    b. Should HealthTrack discontinue support for poorly reviewed models?
    c. Is there an opportunity to partner with high-performing device brands?

2. Justify why a NoSQL database solution will solve the identified business problem.

A NoSQL database will provide HealthTrack with a **high-performing, scalable, and flexible** solution. This database will be capable of managing the fast-growing volume and variety of health-related data.

NoSQL databases offer **high performance** with real-time data processing and low latency. Their architecture enables efficient read and write operations across multiple nodes, making them ideal for high-throughput applications like HealthTrack. This capability will support real-time trend detection and enable the timely delivery of personalized health insights. (InterSystems, n.d.).

NoSQL databases support **horizontal scaling** through built-in sharding, allowing workloads to be distributed across multiple servers. This scalability will allow HealthTrack to grow with its user base, accommodate higher data volumes, and maintain responsiveness even as data from new device types and patient sources are introduced.

The **schema-less design** of NoSQL databases allows them to store and process heterogeneous data formats—for example, sensor readings, text notes, structured EHRs, and user reviews. NoSQL's flexibility will allow HealthTrack to integrate data from various sources without requiring costly schema redesigns. While also making it easier to change the data models as business needs change or new device features are introduced.

3. Identify a NoSQL database type to solve the identified business problem.

For HealthTrack's application, a document-oriented NoSQL database designed in MongoDB is the most appropriate solution. While NoSQL databases support a variety of data models (key-value, column-family, graph), the document store model is best suited for HealthTrack's diverse and dynamic data.

Document stores organize and store data in self-describing documents, typically in JSON, BSON, or XML formats. MongoDB natively supports JSON-like documents, making it an ideal match for HealthTrack's incoming data streams, which are already structured in JSON format.

A document-based model gives HealthTrack the flexibility to handle data collection changes without constantly restructuring the database. MongoDB makes this even easier with built-in indexing, powerful search tools, and the ability to scale across multiple servers. These features are key to running complex analytics smoothly as the platform grows.

4. Explain how the business data will be used within the database solution.

I have divided HealthTrack's data into two main collections: medical and fitness_trackers. Each is tailored to support how the platform processes and makes sense of user data, helping HealthTrack better understand its users and make smarter business decisions.

Medical collection:

This collection will store structured and semi-structured health-related data for each user, such as patient_id, name, date_of_birth, gender, medical_conditions, medications, allergies, last_appointment_date, and Tracker.

By analyzing data in this collection, HealthTrack can:

- Identify trends in patient health across populations.
- Determine whether healthier individuals are more likely to use tracking devices.
- Develop predictive models for personalized health recommendations.

Fitness_trackers collection:

This collection will store metadata about the tracking devices, including Brand Name, Device Type, Model Name, Color, Selling Price, Original Price, Display, Rating (Out of 5), Strap Material, Average Battery Life (in days), and Reviews.

By analyzing data in this collection, HealthTrack can:

- Determine which brands or models are most and least preferred.
- Assess correlations between device cost and customer satisfaction.
- Make informed decisions about discontinuing support for low-performing devices or pursuing partnerships with high-performing brands.

B. Discuss how the proposed database design addresses scalability concerns, including strategies that align with the chosen scenario.

The proposed Document Store NoSQL database, implemented in MongoDB, addresses HealthTrack's scalability concerns through horizontal scaling, sharding, and replication. Allowing the database to efficiently handle larger amounts of data and adapt to changing data structures.

Horizontal Scaling (Scale-Out Architecture):

MongoDB supports horizontal scaling, allowing the system performance to grow by adding additional nodes to the database cluster. As HealthTrack's user base and real-time data inputs expand,

this architecture allows the platform to distribute workloads across multiple machines and maintain consistent performance under heavy demand.

Sharding (Data Partitioning):

MongoDB uses sharding to partition large datasets into smaller, more manageable segments called “shards”. These shards are distributed across nodes within the cluster. This enables HealthTrack to process queries and ingest real-time data in parallel, increasing throughput and reducing response times. Sharding can also help isolate workloads. This will enhance performance across both collections.

Replication (Availability and Fault Tolerance):

MongoDB improves reliability by replicating data across multiple servers. If one server goes down due to a hardware or other technical problem, the system can automatically switch to a backup without losing access. This keeps the platform running smoothly and avoids disruptions, which is significant for HealthTrack, where users need constant access to their health data.

C. Outline the privacy and security measures that should be implemented in the proposed database design.

Given the sensitive nature of healthcare data, the proposed MongoDB database design must strictly adhere to privacy and security standards, particularly those outlined in the Health Insurance Portability and Accountability Act (HIPAA). If HealthFit doesn’t follow these regulations, it could face fines, legal trouble, and a serious hit to its reputation and the trust of its users. To help keep their data safe and stay compliant, HealthFit should focus on the following steps:

Data Encryption:

All production data must be encrypted at rest and in transit using industry-standard protocols.

Access Control and Multi-Factor Authentication (MFA):

Access to the database should be strictly limited based on role-based access control (RBAC), and all users must authenticate using MFA to mitigate the risk of credential-based breaches.

Regular Security Audits and Penetration Testing:

Internal and external third-party security teams must regularly perform testing to identify vulnerabilities.

MongoDB Native Security Features:

MongoDB provides a comprehensive suite of built-in security capabilities that HealthTrack can deploy on-premises or through MongoDB Atlas. These capabilities include authentication, authorization, auditing, encryption, network isolation, and data sovereignty tools (MongoDB, n.d.). Leveraging these features will allow HealthFit Innovations to maintain a strong security posture while meeting HIPAA compliance requirements.

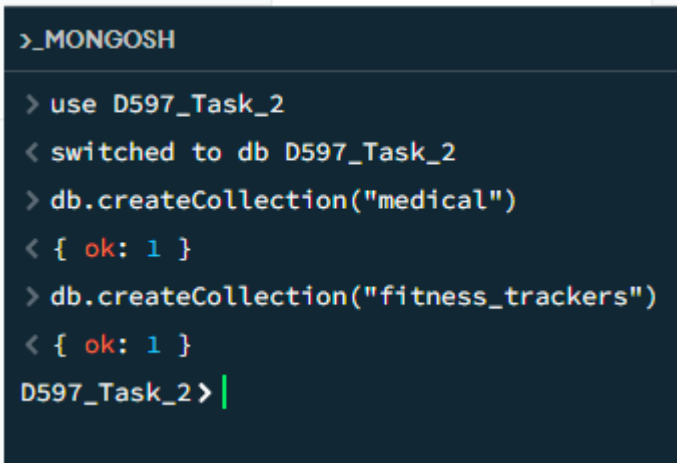
Part 2: Implementation

D. Implement the proposed database design in the WGU Virtual Lab environment by completing the following:

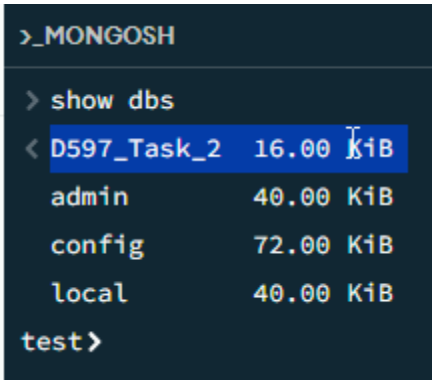
1. Write script to create a database instance named "D597 Task 2" using the appropriate query language, based on your design in Part 1. Provide a screenshot showing the script and the database instance in the platform.

The following script was executed in the **MongoDB Shell (mongosh)** to create the new database D597_Task_2 along with its associated collections based on the document store design.

```
use D597_Task_2

db.createCollection("medical")

db.createCollection("fitness_trackers")
```



In MongoDB the "use" command switches to the specified database context and creates the database once at least one collection is added. In the provided screenshots, we can see that both collections were added – medical and fitness_trackers.



2. Write script to insert or map the data records from the chosen scenario JSON files into the database instance. Provide a screenshot showing the script and the data correctly inserted or mapped into the database.

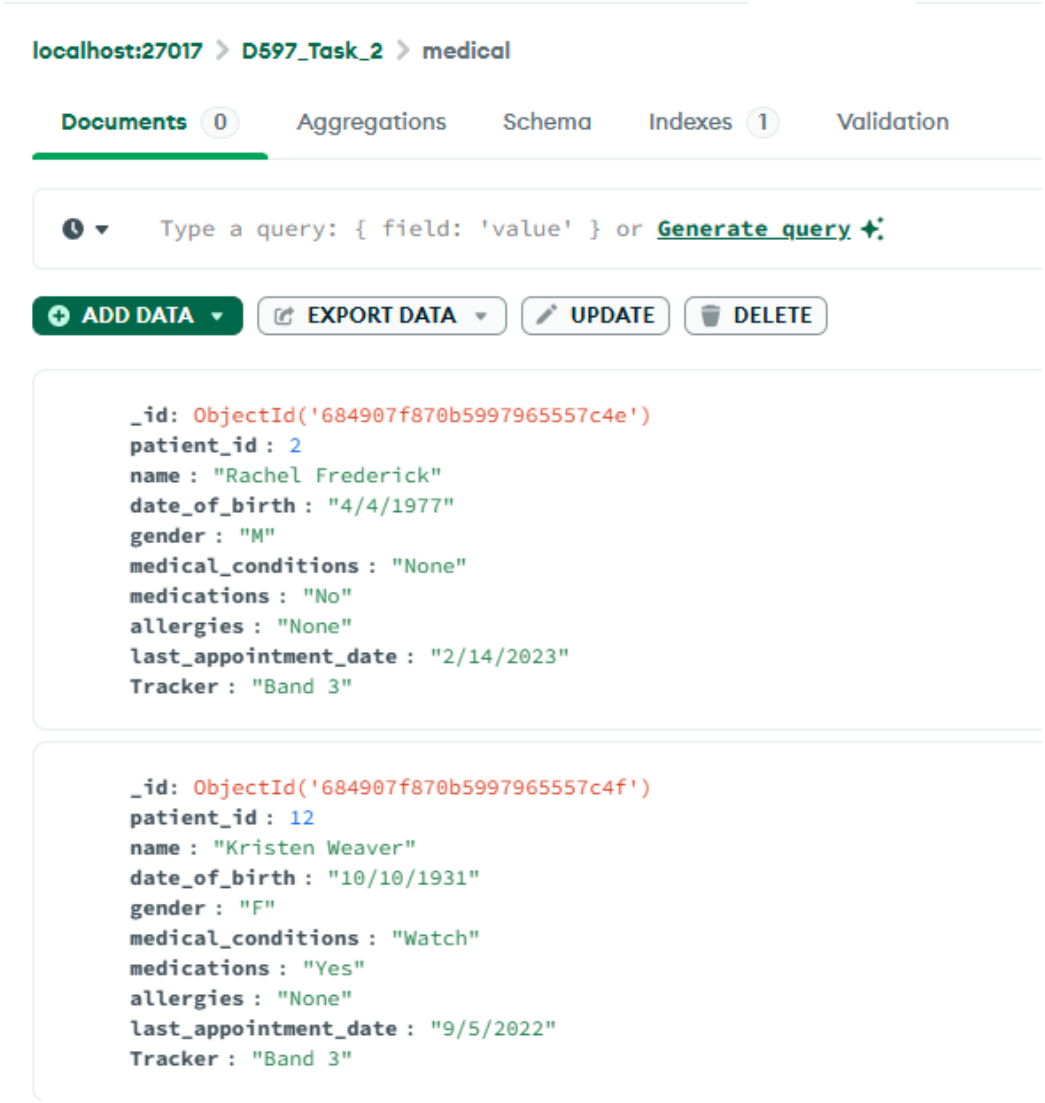
Using the **mongoimport command-line utility**, data from the provided scenario JSON files was imported into the D597_Task_2 database. The data was organized into two collections – medical and fitness_trackers.

```
//import the medical data into the medical collection
mongoimport --uri "mongodb://localhost:27017" ^
  --db D597_Task_2 ^
  --collection medical ^
  --file "C:\Users\PaceW\Desktop\WGU\Courses\D597 - Data Management\Task2\Task 2 Scenario 1\Task 2 Scenario 1\medical.json" ^
  --jsonArray


//import the fitness_trackers data into the fitness_trackers collection
mongoimport --uri "mongodb://localhost:27017" ^
  --db D597_Task_2 ^
  --collection fitness_trackers ^
  --file "C:\Users\PaceW\Desktop\WGU\Courses\D597 - Data Management\Task2\Task 2 Scenario 1\Task 2 Scenario 1\Task2Scenario1 Dataset 1_fitness_trackers.json" ^
  --jsonArray
```

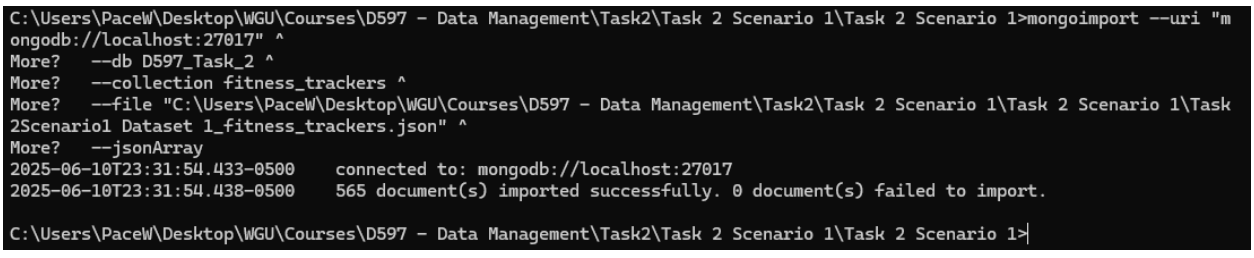
Mongoimport script executed for medical. **100,000 documents were imported successfully, 0 documents failed to import**.

```
C:\Users\PaceW\Desktop\WGU\Courses\D597 - Data Management\Task2\Task 2 Scenario 1\Task 2 Scenario 1>mongoimport --uri "m
ongodb://localhost:27017" ^
More?   --db D597_Task_2 ^
More?   --collection medical ^
More?   --file "C:\Users\PaceW\Desktop\WGU\Courses\D597 - Data Management\Task2\Task 2 Scenario 1\Task 2 Scenario 1\medi
cal.json" ^
More?   --jsonArray
2025-06-10T23:37:12.800-0500    connected to: mongodb://localhost:27017
2025-06-10T23:37:13.577-0500    100000 document(s) imported successfully. 0 document(s) failed to import.

C:\Users\PaceW\Desktop\WGU\Courses\D597 - Data Management\Task2\Task 2 Scenario 1\Task 2 Scenario 1>
```

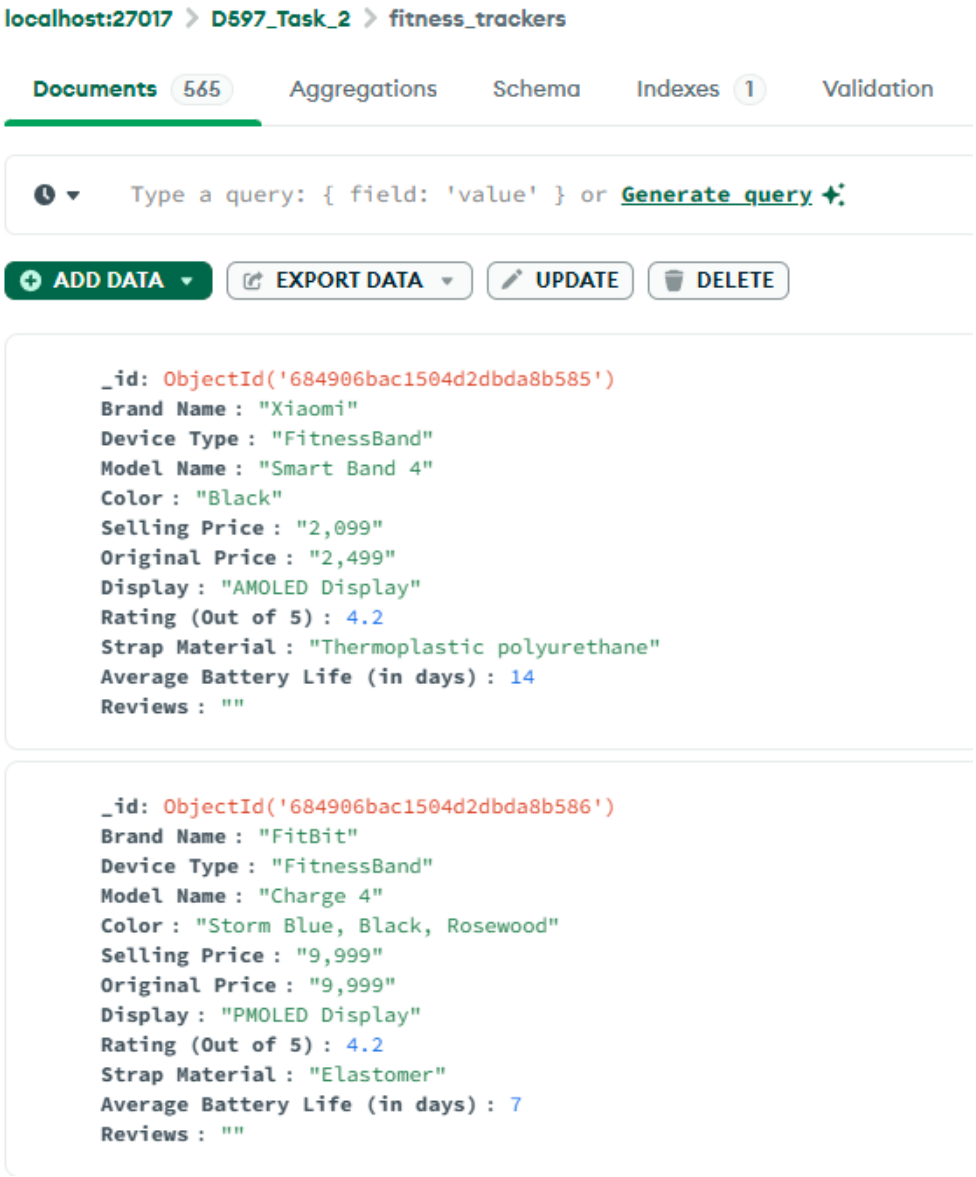
Sample records in the medical collection within MongoDB Compass.



Mongoimport script executed for fitness_trackers. **565 documents imported successfully, 0 documents failed to import.**



Sample records in the fitness_trackers collection within MongoDB Compass



3. Write script for three queries to retrieve specific information from the database that will help to solve the identified business problem. Provide a screenshot showing the script for each query and each query successfully executed.

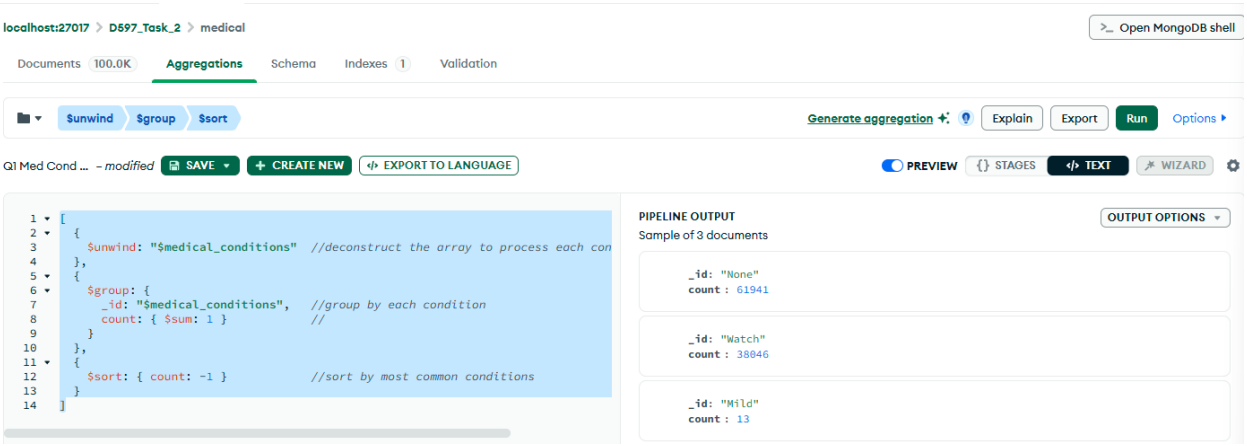
Script 1: Understanding Health Trends Among Users

This will help determine if there is a pattern in medical conditions among users who use tracking devices. Specifically, it will count how many users fall into each category of medical condition by unwinding the medical_conditions array, grouping by condition type, and sorting by frequency.

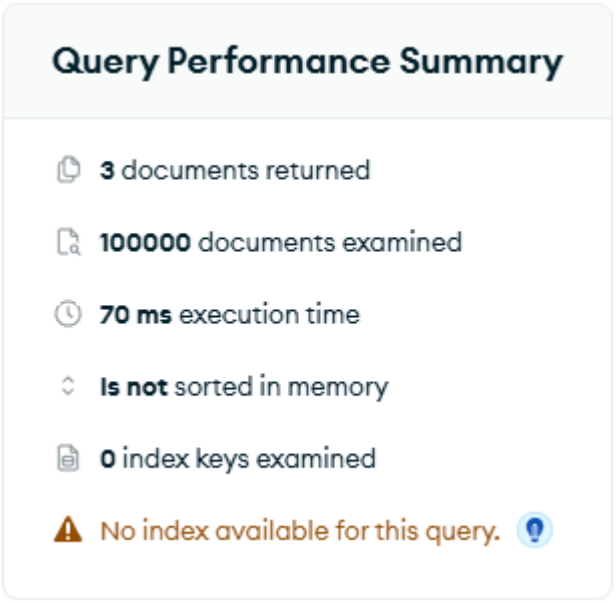
Here is the original script, an aggregation pipeline query written in Mongosh.

```
//query 1 what is the medical condition of patients with trackers?
db.medical.aggregate([
  {
    $unwind: "$medical_conditions"  //deconstruct the array to process each condition separately
  },
  {
    $group: {
      _id: "$medical_conditions",    //group by each condition
      count: { $sum: 1 }
    }
  },
  {
    $sort: { count: -1 }            //sort by most common conditions
  }
]);
```

Query results show the most common medical conditions among patients with tracking devices. A majority report “None”. This insight supports the hypothesis that healthier individuals may be more likely to proactively use fitness tracking devices. This is valuable for marketing, product targeting, and user engagement strategies.



The query examined 100,000 documents and executed in 70 milliseconds.

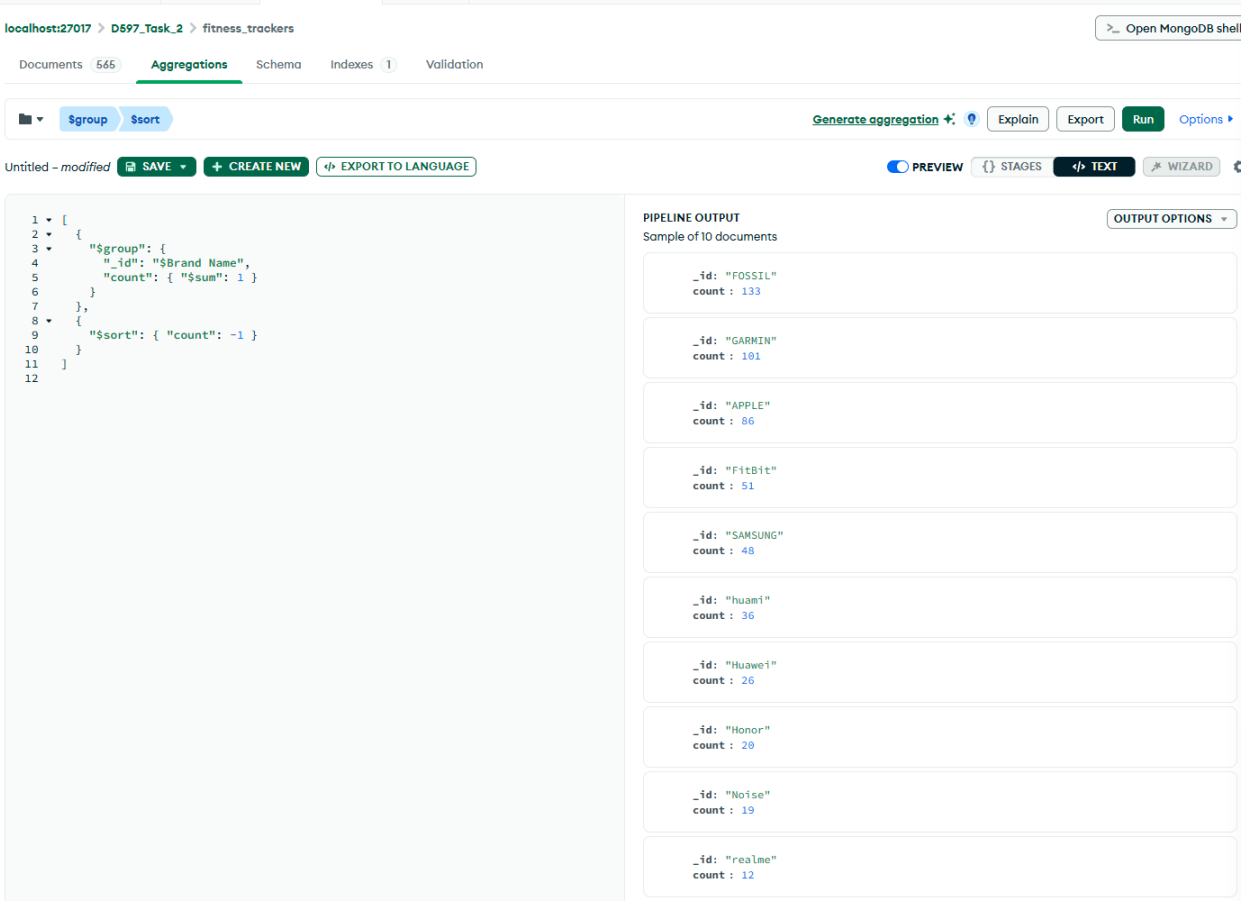


Script 2: Device Brand Preference and Support Strategy

This query helps identify brand preference among HealthTrack users. HealthFit Innovations can make informed decisions about device support, compatibility prioritization, and potential partnership opportunities with this information.

```
//query 2 identify the most popular and least popular brands
db.fitness_trackers.aggregate([
  { $group: {
      _id: "$Brand Name",        // Group by the 'Brand Name' field
      count: { $sum: 1 }         // Count each occurrence
    }
  },
  { $sort: { count: -1 }         // Sort by count in descending order
  }
])
```

The results show Fossil (133), Garmin (101), and Apple (86) are the most frequently used brands in the dataset, indicating a strong user preference. Conversely, LAVAL, Infinix, and Oppo are among the least common. These insights indicate that HealthFit Innovations can research why some of these brands are so unpopular and determine if there is value in maintaining support. Also, this insight can inform decisions on prioritizing feature enhancements or partnerships with top-performing brands.




```
_id: "OnePlus"
count : 3
```

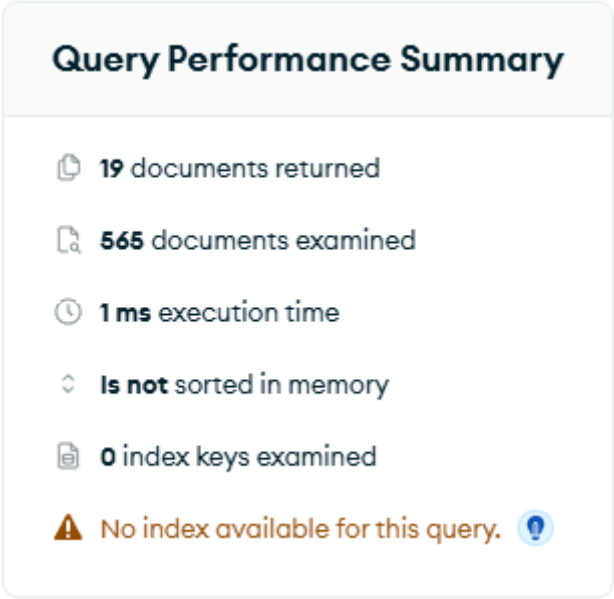
```
_id: "LCARE"
count : 2
```

```
_id: "Oppo"
count : 2
```

```
_id: "Infinix"
count : 1
```

```
_id: "LAVA"
count : 1
```

The query was executed in 1 millisecond across 565 documents.



Script 3: Product Price and Customer Sentiment

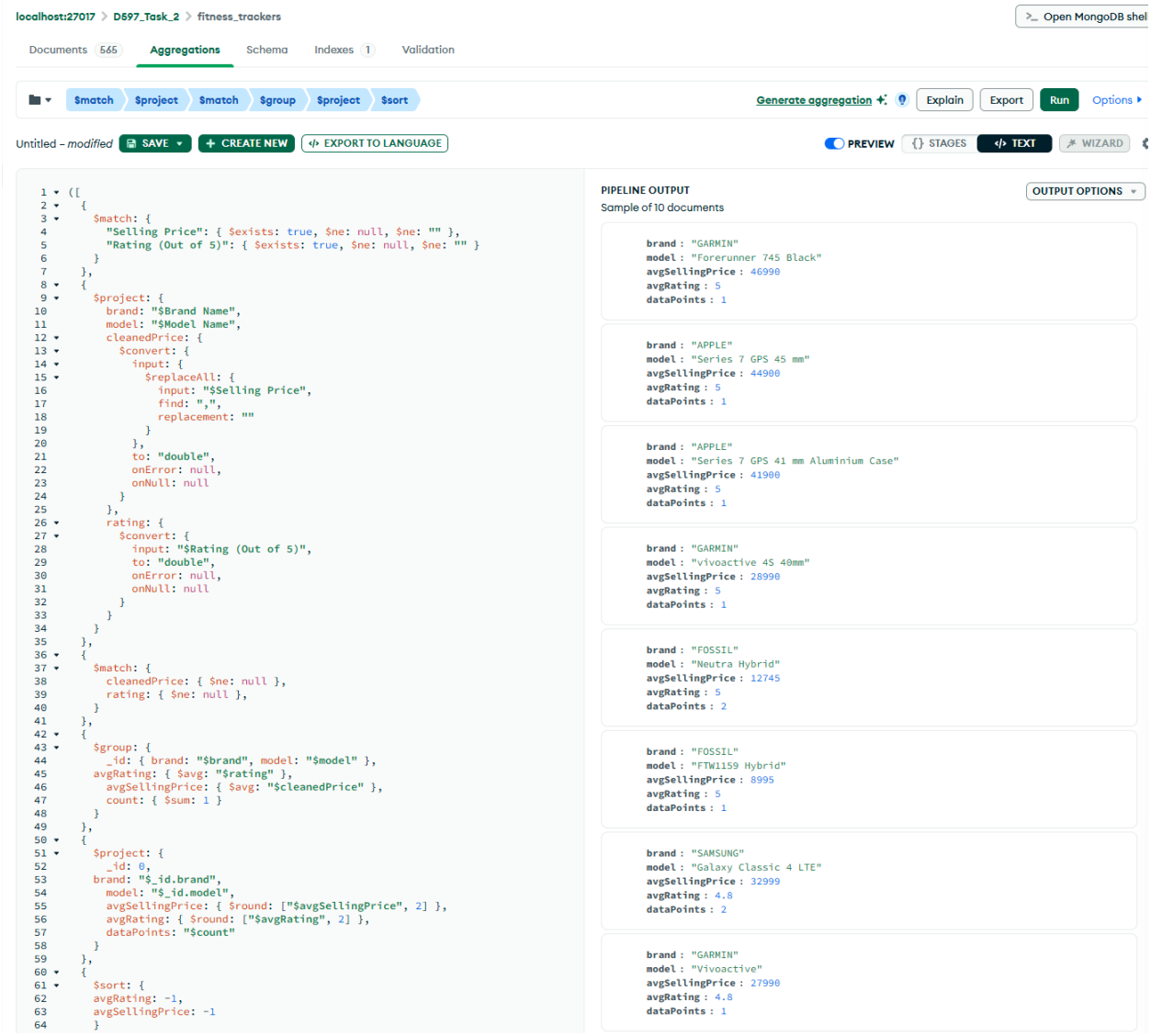
This query helps determine whether more expensive fitness trackers receive higher customer ratings.

We clean the selling prices and rating fields by converting them from strings to numeric types. Then filter out documents with missing, null, or invalid values. Then group the results by device brand and model. Compute the average selling prices and average ratings. Lastly, sort the results by rating and price in descending order.

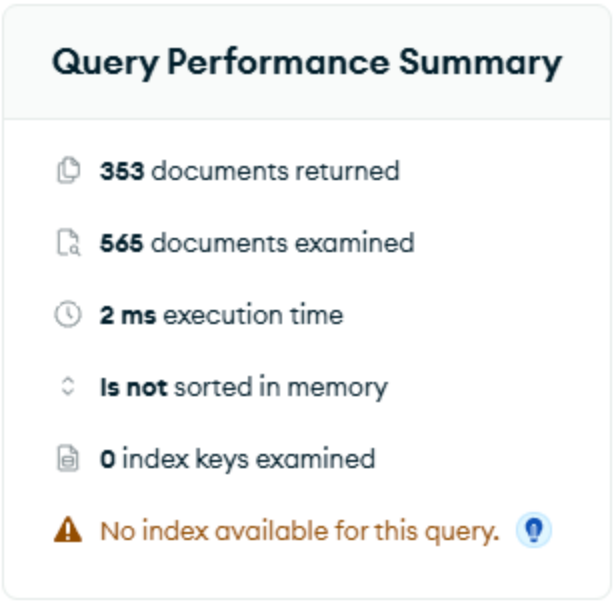
```
//query 3 Do the more expensive models have higher customer reviews?
db.fitness_trackers.aggregate([
  {
    $match: {                   //filter out documents with missing, null, or empty selling prices or ratings
      "Selling Price": { $exists: true, $ne: null, $ne: "" },
      "Rating (Out of 5)": { $exists: true, $ne: null, $ne: "" }
    }
  },
  {
    $project: {                 //clean and convert string fields to numbers
      brand: "$Brand Name",
      model: "$Model Name",
      cleanedPrice: {           //convert selling price from a string to a double
        $convert: {
          input: {
            $replaceAll: {
              input: "$Selling Price",
              find: ",",
              replacement: ""
            }
          },
          to: "double",
          onError: null,        //error handling
          onNull: null
        }
      },
      rating: {                 //convert ratings from string to double
        $convert: {
          input: "$Rating (Out of 5)",
          to: "double",
          onError: null,
          onNull: null
        }
      }
    }
  },
  {
    $match: {               //remove records with invalid values
      cleanedPrice: { $ne: null },
      rating: { $ne: null },
    }
  },
  {
    $group: {               //group by model, compute averages and totals
      _id: { brand: "$brand", model: "$model" },
      avgRating: { $avg: "$rating" },
      avgSellingPrice: { $avg: "$cleanedPrice" },
      count: { $sum: 1 }
    }
  },
  {
    $project: {             //clean output
      _id: 0,
      brand: "$_id.brand",
      model: "$_id.model",
      avgSellingPrice: { $round: ["$avgSellingPrice", 2] },
      avgRating: { $round: ["$avgRating", 2] },
      dataPoints: "$count"
    }
  },
  {
    $sort: {                //sort by price (high low), then ratings (high low)
       avgRating: -1,
       avgSellingPrice: -1
       }
  }
]);
```

Sample output showing the brand/model with associated average price and rating. These results suggest that prices are not always correlated with a higher customer rating. For example, some of the mid-priced models – Fossil and Garmin – received equally high or higher ratings compared to the more expensive options like Apple or Samsung. This information can be leveraged in strategic decisions about

which devices to prioritize for partnerships, marketing, and continued support.



The query examined 565 documents and returned 353 results in just 2 milliseconds.

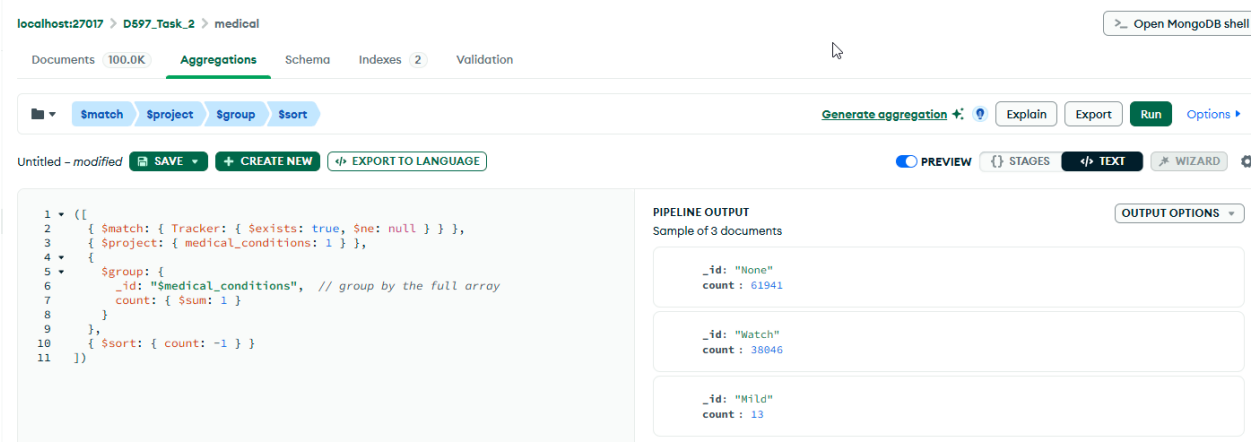


4. Apply optimization techniques to improve the run time of your queries from part D3, providing output results via a screenshot.

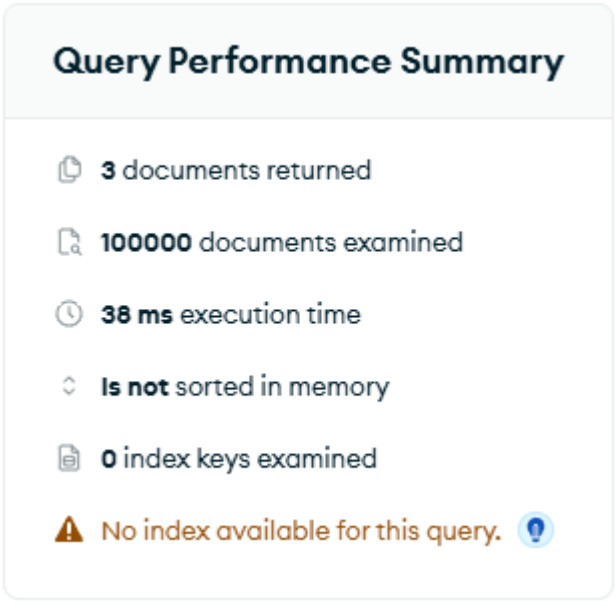
Query 1 optimization

In the original query, the $unwind operator was used to deconstruct the medical_conditions array for analysis. This allows for granular insight but introduces higher CPU and memory usage.

To optimize the query, the $unwind was removed and replaced with a combination of $match, $project, and $group. Allowing analysis of medical conditions at the document level rather than the element level. This adjustment reduced computation without sacrificing the integrity of the grouped results.



Execution time improved from 70ms to 38ms while examining 100,000 documents. Achieve a 46% performance improvement by limiting the aggregation workload to top-level fields and avoiding resource-intensive operators when unnecessary.



Query 2 optimization

Originally, the aggregation pipeline grouped documents by “Brand Name” to determine the most and least popular tracker brands. This query was already fast at 1ms; however, it examined all documents because there was no index.

To optimize this query, the following improvements were made:

- Created an ascending index on the “Brand Name” field.
- Applied $match to filter out any null or malformed values early.
- Used the $project to streamline documents before aggregation.

Index creation, index Brand Name_1, was created successfully.

```
> db.fitness_trackers.createIndex ({"Brand Name" : 1})
< Brand Name_1
D597_Task_2>
```

Optimized version of the script successfully executed in MongoDB Compass.

```
[
  {
    $match: {
      "Brand Name": {
        $exists: true,
        $ne: null,
        $ne: ""
      }
    }
  },
  {
    $project: {
      brand: "$Brand Name"
    }
  },
  {
    $group: {
      _id: "$brand",
      count: {
        $sum: 1
      }
    }
  },
  {
    $sort: {
      count: -1
    }
  }
]
```

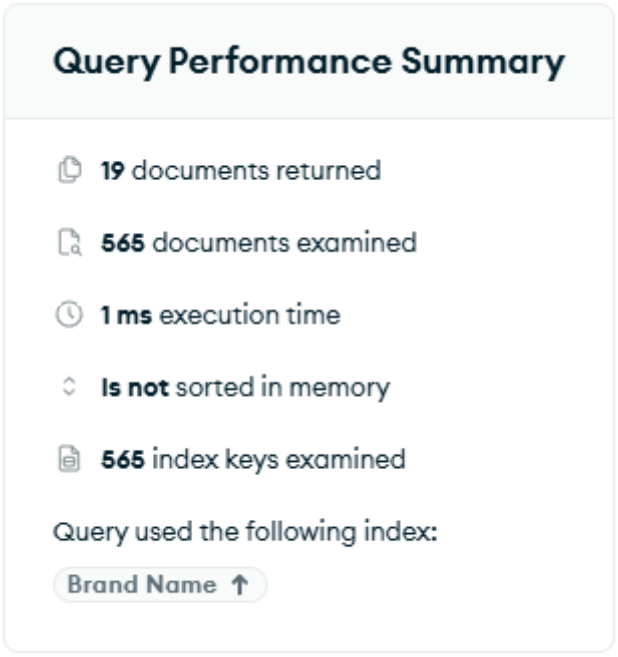
PIPELINE OUTPUT

OUTPUT OPTIONS

Sample of 10 documents

```
_id: "FOSSIL"
count : 133

_id: "GARMIN"
count : 101

_id: "APPLE"
count : 86

_id: "FitBit"
count : 51

_id: "SAMSUNG"
count : 48

_id: "huami"
count : 36
```

While MongoDB correctly utilized the index to filter documents (565 index keys examined), the aggregation’s $group stage still required scanning every document to count brand occurrences. As a result, execution time remained at 1 millisecond. This optimization serves future-proof performance as the dataset scales.



Query 3 optimization

This query involved numerical operations and aggregation on the Selling Price and Rating (out of 5) fields to determine if higher-priced models receive better reviews. To optimize performance, a compound index was created on both fields, which will be used in the $match stage of the query.

A compound index is created on Selling Price and Rating (Out of 5).

```
> db.fitness_trackers.createIndex({ "Selling Price": 1, "Rating (Out of 5)": 1 })
< Selling Price_1_Rating (Out of 5)_1
D597_Task_2>
```

The index was used, and we see a reduction in the number of documents examined, from 565 to 514. Although execution time remained at 3ms, this is a decrease of 9% in the number of documents examined. This suggests that the index was effective in narrowing the dataset for evaluation. This reduction would likely lead to noticeable performance gains and reduced server load on larger datasets.

Query Performance Summary

- **353** documents returned
- **514** documents examined
- **3 ms** execution time
- **Is not** sorted in memory
- **546** index keys examined

Query used the following index:

Selling Price ↑

Rating (Out of 5) ↑

References

InterSystems. (n.d.). NoSQL databases explained: Advantages, types, and use cases. https://www.intersystems.com/resources/nosql-databases-explained-advantages-types-and-use-cases/

MongoDB. (n.d.). Security: Features and capabilities. https://www.mongodb.com/products/capabilities/security